



School of Computing and Engineering Sciences

BBT 2202: Advanced Object-Oriented Programming

Lab Practical Assessment: OOP Concepts – Inheritance & Polymorphism

220284 – Muindi George Sila

Instructions

1. This is an individual assignment; you can work in pairs but submit different work.
2. Answer every question on **this document** in the spaces provided.
3. Maintain the document format when answering EXCEPT when writing code.
4. Provide additional information where you feel diagrams may not be enough.
5. Once you are done answering, ***convert the document to .pdf format*** for submission.

Objective:

- Understand inheritance and polymorphism in Java and their benefits.
- Create a parent (superclass) and a child (subclass).
- Use the extends keyword to establish inheritance.
- Override methods in a subclass.
- Use super to call parent class constructors, attributes or methods.
- Use NetBeans IDE to write and run Java programs.

Prerequisites:

- NetBeans 28 (or later) installed on your computer.
- Basic understanding of programming concepts and Encapsulation.

Tutorial

Part A – Inheritance and Polymorphism

Step 1: Open NetBeans IDE

1. Launch NetBeans IDE.
2. Click File → New Project.
3. Select Java with Ant → Java Application and click Next.
4. Name the project '**OOPConceptsDemo**'.
5. Uncheck “create main class”, then click Finish."

Step 2: Create the **Student** class (***Parent Class***).

1. Right-click on the Source Packages folder in the Projects window.

2. Select New → Java Package.
3. Name the package '**inheritancePolymorphism**' and click finish.
4. Right-click on the '**inheritancePolymorphism**' package.
5. Select New → Java Class.
6. Name the class '**Student**' and click Finish.

Step 3: Write **code for the Student** Class

1. ***Challenge yourself to write the code without looking at the solution at the end***
2. Declare three attributes that a student can have (**studentId**, **studentName**, **studentAge**). The attributes should be hidden but accessible by a child/sub class (declared as protected) and initialized to hold null or 0.
3. Implement getters for each of the attributes (remember to name them appropriately using the "get-" prefix). (**Hint:** getters return data and have that as the implementation, no parameters)
4. Implement setters for each of the attributes (remember to name them appropriately using the "set-" prefix). (**Hint:** setters do not return data but instead, assign a value obtained from its parameter to the respective attribute. Parameter has the same datatype as attribute).
5. Implement a method '**introduce()**' that when invoked, it will output the student details.
6. Implement a parameterised constructor for the class.

If the steps on hidden attributes do not make sense, revisit the notes on encapsulation and consider completing the optional lab tutorial on encapsulations.

Step 4: Create the **GraduateStudent** class (Child class to the Student super class)

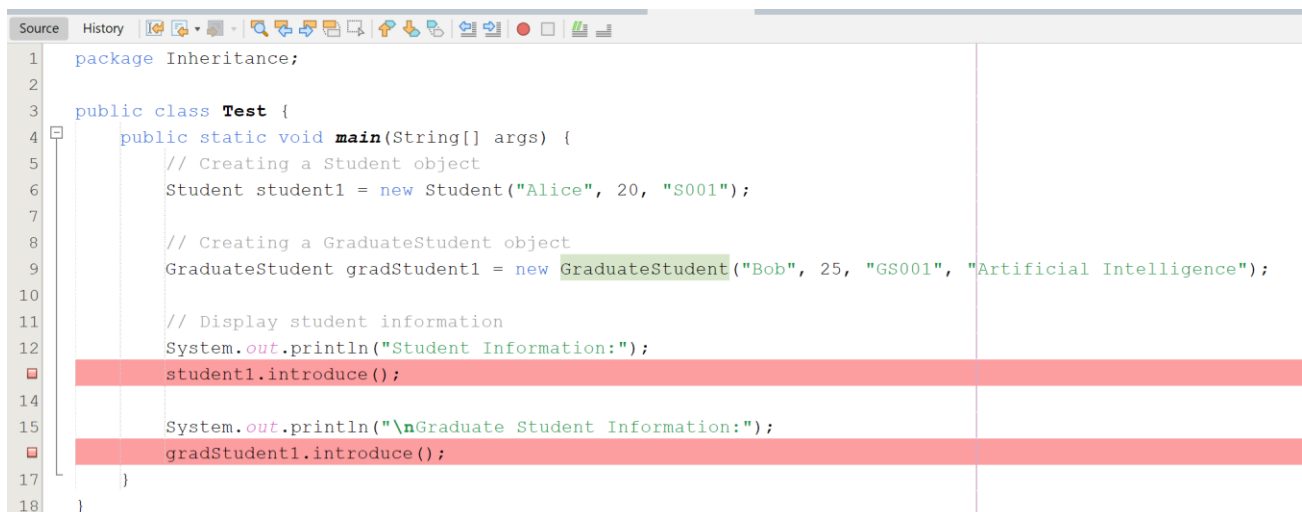
1. Right-click on the '**Inheritance**' package.
2. Select New → Java Class.
3. Name the class '**GraduateStudent**' and click Finish.

Step 5: Write **code for the GraduateStudent** Class

1. *Challenge yourself to write the code without looking at the solution at the end*
2. Declare a hidden String variable to hold the graduate student's research topic ('researchTopic').
3. Implement a constructor that will *initialize all the attributes* of a graduate student. (**Hint:** use the super keyword to call the constructor of the parent class)
4. Implement a method 'introduce()' that when invoked, will output **all** the graduate student details, including the research topic. (**Hint:** Method overriding and using the "super" keyword to call attributes and methods)

Step 5: Create the 'main' or 'test' class

1. Right-click on the 'Inheritance' package.
2. Select New → Java Class.
3. Name the class 'Test' and click Finish.
4. Write the code shown in the screenshot that follows:



```

1 package Inheritance;
2
3 public class Test {
4     public static void main(String[] args) {
5         // Creating a Student object
6         Student student1 = new Student("Alice", 20, "S001");
7
8         // Creating a GraduateStudent object
9         GraduateStudent gradStudent1 = new GraduateStudent("Bob", 25, "GS001", "Artificial Intelligence");
10
11         // Display student information
12         System.out.println("Student Information:");
13         student1.introduce();
14
15         System.out.println("\nGraduate Student Information:");
16         gradStudent1.introduce();
17     }
18 }

```

Notice different objects calling the same method introduce; will the output be the same?

Step 5: Run the code

1. Right-click on the **Test.java** file.
2. Click Run File.
3. Observe the output in the Output Window for the following.

```
Output - OOPConceptsDemo (run) x
run:
Student Information:
Student ID: S001
Student Name: Alice
Student Age: 20

Graduate Student Information:
Student ID: GS001
Student Name: Bob
Student Age: 25
Research Topic: Artificial Intelligence
BUILD SUCCESSFUL (total time: 0 seconds)
```

Why is the output for the different objects different yet they both invoked the same method introduce?

Part B – Questions

1. Explain your understanding of inheritance after having completed this tutorial.

Inheritance is an object-oriented programming concept where one class (the child or subclass) derives attributes and methods from another class (the parent or superclass). It promotes code reuse, reduces duplication, and allows us to extend existing functionality without rewriting everything. For example, GraduateStudent inherits from Student, so it automatically has all the student attributes and behaviors, plus its own specialized ones..

2. Explain some of the applications of inheritance in software development (**Hint:** *reusability, maintainability and modularity, extensibility and modularity among others*).
 - **Reusability:** Common code is written once in a base class and reused in subclasses.
 - **Maintainability:** Changes in the parent class automatically propagate to subclasses, reducing maintenance effort.
 - **Modularity:** Systems can be broken into logical hierarchies, making them easier to understand and manage.
 - **Extensibility:** New features can be added by creating subclasses without altering existing code.
 - **Polymorphism:** Subclasses can override methods, allowing flexible and dynamic behavior at runtime.

3. What type of inheritance does java not support?

Java does not support **multiple inheritance with classes** (a class cannot extend more than one parent class). This restriction avoids ambiguity problems

4. Overload (*polymorphism*) the introduce method such that an extra detail which is taken as a parameter to be output when the method is called. The extra detail can be the course:

```
// Overloaded introduce method
public void introduce(String course) {

    System.out.println("Student ID: " + studentId);
    System.out.println("Name: " + studentName);
    System.out.println("Age: " + studentAge);
    System.out.println("Course: " + course);

}
```

5. Modify the Main class to set and retrieve **studentCourse**.

```
public class Test {

    public static void main(String[] args) {
        Student student1 = new Student("Alice", 20, "S001");
        student1.introduce("Database Systems"); // using overloaded method
    }

}
```

Experiment with the code (you can create another child class called ‘UndergraduateStudent’ and practice on Inheritance, submit this in a separate pdf document)

Code for the **Student** class before modification

Package inheritance;

```
// Student.java
public class Student {

    // Private fields (Encapsulation)
    private String studentId;
    private String studentName;
    private int studentAge;

    // Constructor
    public Student(String name, int age, String id){
        this.studentName = name;
        this.studentAge = age;
        this.studentId = id;
    }

    // Getter methods
    public String getStudentName() {
        return studentName;
    }

    public int getStudentAge() {
        return studentAge;
    }

    public String getStudentID() {
        return studentId;
    }

    // Setter methods
    public void setStudentName(String name){
        this.studentName = name;
    }

    public void setStudentAge(int age){
        this.studentAge = age;
    }

    public void setStudentID(String id){
        this.studentId = id;
    }

    // Display student details
    public void introduce() {
        System.out.println("Student ID: " + studentId);
        System.out.println("Student Name: " + studentName);
        System.out.println("Student Age: " + studentAge);
    }
}
```

Code for the **GraduateStudent** class before modification

```
package inheritance;

// Child class (Subclass)
public class GraduateStudent extends Student {
    private String researchTopic ;

    // Constructor calling the superclass constructor
    public GraduateStudent( String name, int age, String id, String topic) {
        super(name, age, id); // Calls the constructor of Student
        this.researchTopic = topic;
    }

    // Overriding method to display student details
    @Override
    public void introduce() {
        super.introduce(); // Call parent method
        System.out.println("Research Topic: " + researchTopic);
    }
}
```